

COLLECTORHUB

Bitácora de Seguimiento del Proyecto

Fecha: 18 de marzo de 2026

Objetivo: Construcción del Prototipo Mínimo Funcional (MVP) y despliegue de la infraestructura base (Hito 1).

1. Infraestructura y Contenedores (Docker)

Se ha iniciado la jornada estableciendo el entorno de ejecución para garantizar la portabilidad del proyecto en Windows 11.

- **Tarea:** Configuración y despliegue de **Docker Desktop** mediante el motor **WSL 2**.
- **Implementación:** Creación de un archivo `docker-compose.yml` para orquestar un contenedor de **MySQL 8.0**.
- **Persistencia:** Se han configurado volúmenes de datos para asegurar que la información de la base de datos no se pierda al reiniciar los contenedores.

2. Diseño y Automatización de la Base de Datos

Se ha definido la estructura relacional del proyecto siguiendo los requerimientos del módulo de inventario.

- **Tarea:** Creación del script de inicialización `init.sql`.
- **Modelo de Datos:** Implementación de las tablas maestras:
 - **usuarios:** Con soporte para alias, roles (RBAC) y contraseñas cifradas.
 - **categorías:** Incorporación del campo **JSON esquema** para permitir la definición dinámica de atributos por parte del usuario.
 - **articulos:** Vinculación con usuarios y categorías, incluyendo campos **JSON** para metadatos específicos y gestión de múltiples fotografías.

Autor: Joaquín Bizcocho

- **Automatización:** Configuración de la carpeta `mysql-init` para que Docker ejecute el esquema y los datos de prueba (`profe_test`) automáticamente al arrancar.

3. Desarrollo del Backend (Spring Boot)

Se ha construido el "puente" de comunicación entre los datos y la interfaz de usuario utilizando IntelliJ IDEA.

- **Tarea:** Inicialización del proyecto Spring Boot con dependencias de **Spring Web, JPA y MySQL Driver**.
- **Resolución de Incidencias:** Se ha identificado y resuelto un error crítico de autenticación JDBC (`auth_gssapi_client`) mediante la reconfiguración de la URL de conexión en `application.properties`, forzando el uso de `allowPublicKeyRetrieval` y el dialecto de Hibernate para MySQL.
- **Lógica de Negocio:** Creación del `LoginController` para validar credenciales. Se ha verificado el éxito del login mediante pruebas con archivos `.http` internos.

4. Desarrollo del Frontend (React + Vite)

Se ha iniciado la construcción de la interfaz visual en Visual Studio Code para ofrecer una experiencia de usuario real.

- **Tarea:** Creación del proyecto mediante **Vite** y desarrollo de la arquitectura de componentes.
- **Funcionalidades:** * Formulario de acceso conectado por `fetch` al puerto `8080` del backend.
 - Gestión de estados para el manejo de credenciales y mensajes de respuesta.
- **Mejoras de UX:** Implementación de la función "Mostrar/Ocultar contraseña" mediante el uso de un botón con estados booleanos.

5. Estética y Diseño de Interfaz (UI)

Se ha aplicado una capa de estilos personalizada para dotar al proyecto de una identidad visual profesional.

- **Paleta de Colores:** Selección de una gama de **verdes suaves** sobre un fondo blanco puro para transmitir limpieza y orden.

- **Composición:** Centrado absoluto de la tarjeta de login en pantalla completa.
- **Refinamiento:** Ajuste de sombras (box-shadow), bordes redondeados y transiciones suaves para mejorar la interactividad de los botones y campos de texto.

Fecha: 19 de marzo de 2026

Objetivo: Finalización, pulido, documentación y empaquetado del Prototipo Mínimo Funcional (Hito 1).

1. Preparando el Login para la base de datos real

Me di cuenta de que el login ya funcionaba y daba el mensaje de "Acceso concedido", pero era porque habíamos "hardcodeado" (puesto a mano) el usuario y la contraseña en el código de Spring Boot para hacer la prueba. Hoy he dejado preparado el terreno con JPA (creando la clase `Usuario` y su repositorio) para que en el futuro el programa busque de verdad si el usuario existe en la tabla de MySQL en vez de usar un usuario fijo.

2. Poniendo orden y mejorando el diseño en React

Al principio tenía todo el código del Frontend un poco mezclado. Lo he reestructurado creando una carpeta específica solo para el componente del Login y su propio archivo CSS, así está todo mucho más organizado para cuando tenga que añadir más pantallas. También estuve peleando con un fallo molesto que me dejaba unas líneas finas por los bordes de la pantalla, pero lo solucioné haciendo un "reset" en el CSS global para que el fondo blanco ocupe el 100% de la pantalla sin márgenes.

3. El Modo Oscuro (y que no se borre al recargar)

Añadí un botón en la esquina del login para cambiar entre el modo claro y el modo oscuro usando variables en CSS. El problema era que, si recargabas la página, se borraba y volvía a ponerse blanco. Lo he solucionado usando `localStorage` en React; de esta forma, el navegador "recuerde" la opción que ha elegido el usuario y no te pega un pantallazo en los ojos al darle a F5.

4. Documentando el proyecto

Le he dedicado un buen rato a poner comentarios explicativos en el código (usando el formato JSDoc en React y comentando el CSS) para que quede claro qué hace cada función, cada estado y por qué he posicionado las cosas de cierta manera. También redacté la memoria del Hito 1 explicando por qué he separado el Frontend del Backend (arquitectura desacoplada) y los pasos que hay que seguir para probar el proyecto con las credenciales de prueba.

Autor: Joaquín Bizcocho

5. Peleando con un error de conexión de base de datos

Este ha sido el mayor dolor de cabeza del día. El proyecto me empezó a dar un error rarísimo al arrancar Spring Boot: `Unable to load authentication plugin 'auth_gssapi_client'`. Resulta que fue por haber movido el proyecto a través de Google Drive desde otro ordenador:

- Por un lado, la carpeta se me guardó con un "(1)" y espacios en el nombre, lo que volvía loco a Java al intentar compilar.
- Por otro, como el Docker de MySQL se quedó en el otro PC, mi ordenador estaba intentando conectarse a un puerto equivocado y chocaba con cosas de Windows.

¿Cómo lo solucioné? Moví el proyecto a una ruta limpia en mi disco duro (sin paréntesis ni espacios), le cambié el puerto a Docker en el archivo `docker-compose.yml` (puse el 3307 para que no chocara con nada) y limpié la caché de IntelliJ ejecutando un `clean` e `install` de Maven.

Fecha: 8 de abril de 2026

Objetivo: Migración de la infraestructura a la nube e implementación del sistema de registro con validaciones de seguridad.

1. Migración de la base de datos a la nube (Railway)

Para evitar las restricciones de permisos y los bloqueos de red del ordenador de la empresa, he decidido sacar la base de datos de Docker local y subirla a **Railway**. Ahora el proyecto es mucho más flexible porque puedo trabajar desde cualquier sitio conectándome al servidor remoto. He configurado el `application.properties` con los parámetros de seguridad necesarios (SSL y TCP Proxy) para que Spring Boot se comunique con la nube de forma fluida y sin errores de "handshake".

2. Implementación del Registro con seguridad (BCrypt)

No quería un registro simple que solo guardara datos, así que he integrado **Spring Security**. Ahora, cuando un usuario se registra, su contraseña no se guarda tal cual en la base de datos (lo que sería un peligro), sino que se cifra usando el algoritmo **BCrypt**. También he programado la lógica en el controlador para que el sistema rechace el registro si el alias o el correo electrónico ya existen, asegurando que cada coleccionista sea único en la plataforma.

3. Resolviendo el conflicto de JPA y las barras bajas

Hoy me he topado con un error técnico muy específico de Spring Data JPA. Al intentar buscar si un correo existía usando métodos automáticos, el programa explotaba porque mi base de datos usa barras bajas (`correo_electronico`) y Java prefiere el formato *camelCase*. Lo he solucionado usando la anotación **@Query** en el repositorio; de esta forma, le he dictado yo mismo la consulta SQL a Spring para que no intente adivinarla y funcione correctamente con el nombre real de la columna.

4. Navegación fluida en React sin recargas

En el Frontend, he creado el componente `Register.jsx` y he modificado `App.jsx` para gestionar lo que llamamos "renderizado condicional". He usado un estado de React para que actúe como un interruptor: si el usuario pulsa en "Regístrate", la tarjeta de Login desaparece y aparece la de Registro instantáneamente sin que la página tenga que recargarse. Esto da una sensación de aplicación profesional y rápida.

Autor: Joaquín Bizcocho

5. Encapsulado de estilos y corrección de CSS

Al principio, el Login y el Registro compartían el mismo archivo CSS y los estilos empezaban a "chocar" entre ellos. He dedicado la última parte del día a separar los archivos en `Login.css` y `Register.css`. He renombrado todas las clases (usando prefijos como `login-` y `register-`) para que cada componente tenga su propio estilo independiente. Además, he corregido un fallo en el modo oscuro donde el texto "¿No tienes cuenta?" se quedaba en negro y no se leía sobre el fondo oscuro; ahora cambia a blanco automáticamente al pulsar el botón del tema.

Fecha: 9 de abril de 2026

Objetivo: Implementación del motor de gestión de colecciones y panel de control privado para el usuario.

1. Construcción del motor de categorías (JPA y JSON)

Nada más empezar el día, nos hemos centrado en el Backend para crear la estructura de las categorías. El reto técnico era permitir que cada categoría tuviera campos distintos (como "PSA" para cartas o "Autor" para libros). Lo he solucionado implementando una columna de tipo **JSON** en MySQL y mapeándola en Spring Boot usando la anotación `@JdbcTypeCode(SqlTypes.JSON)`. Esto nos da una flexibilidad total sin tener que crear tablas nuevas cada vez que el usuario quiera inventar una colección diferente.

2. Seguridad y Privacidad: El "DNI" del usuario

He corregido un fallo crítico de privacidad que detectamos: antes, cualquier usuario logueado podía ver las categorías de los demás. Para arreglarlo, he modificado el `LoginController` para que, al acceder, el servidor envíe no solo un mensaje de éxito, sino también el **ID** y el **Alias** del usuario. Ahora, React guarda esos datos en el `localStorage` y los usa para filtrar las peticiones al Backend, garantizando que cada coleccionista solo vea y gestione su propio inventario personal.

3. Rediseño visual del Dashboard y experiencia de usuario

Siguiendo el boceto que planteé, hemos dado un salto de calidad en la interfaz. He creado una **cabecera (topbar)** en un verde más oscuro para diferenciarla del cuerpo de la tarjeta, incluyendo el nombre de la app y un botón de "Cerrar Sesión" funcional. También he implementado un mensaje de bienvenida personalizado con el nombre del usuario y he cambiado la disposición de las categorías: ahora son **tarjetas horizontales alargadas** que se apilan hacia abajo, lo que aprovecha mucho mejor el espacio y se ve más profesional que las tarjetas cuadradas iniciales.

4. Finalización del CRUD: Edición y Borrado dinámico

Para que el usuario tenga el control total, hemos completado el sistema **CRUD**. He añadido botones de acción (editar con un lápiz y borrar con una papelera) en cada tarjeta. Aunque al principio tuvimos un problema porque el sistema "no guardaba" los cambios al editar, lo solucionamos rápidamente añadiendo el método `@PutMapping` que faltaba en el controlador de Java. Ahora el usuario puede añadir o quitar columnas

Autor: Joaquín Bizcocho

de su "esquema" de colección en cualquier momento y los cambios se persisten en la nube de Railway al instante.

5. Gestión de errores y validación de contraseñas

Hemos dedicado un tiempo a depurar un error molesto con el inicio de sesión. Descubrimos que, tras implementar el cifrado **BCrypt**, no podíamos entrar con los usuarios de prueba del `init.sql` porque sus contraseñas estaban en texto plano. Hemos validado que el sistema de encriptación funciona perfectamente creando nuevos usuarios desde la web, asegurando que el flujo de registro-cifrado-login es ahora 100% seguro y a prueba de fallos.

Estado del proyecto: El sistema de gestión de categorías es plenamente funcional, privado y dinámico. La aplicación ya sabe quién es el usuario, qué colecciones tiene y cómo debe ser el "molde" para los objetos que crearemos a continuación.

Fecha: 10 de abril de 2026

Hoy he centrado mis esfuerzos en transformar la estructura básica de la aplicación en una plataforma de gestión completa, robusta y con una experiencia de usuario mucho más pulida. He logrado cerrar el ciclo de gestión de inventario y establecer una jerarquía de administración sólida.

Estos son los avances detallados de la jornada:

1. Optimización del acceso y persistencia de sesión

He comenzado el día solucionando un error que impedía la redirección al dashboard tras el login. He simplificado la lógica de validación en el frontend, dejando de depender de mensajes de texto específicos para centrarme en la existencia de un `usuarioId` válido. Además, he implementado la persistencia de sesión; ahora, aunque recargue la página, el sistema reconoce mi sesión activa y me mantiene en el dashboard, mejorando significativamente la comodidad de uso.

2. Implementación del sistema de artículos e imágenes

He desarrollado el núcleo de la aplicación: el inventario. Ahora cada categoría puede contener artículos propios. He diseñado el backend para que soporte esquemas dinámicos (usando JSON) y he integrado la gestión de imágenes. He decidido utilizar formato Base64 para permitir subir hasta dos fotos por artículo sin necesidad de gestionar servidores de archivos externos, lo que facilita enormemente la portabilidad del proyecto.

3. Mejoras en la interfaz y experiencia visual (UX/UI)

Para darle un acabado profesional a la vista de inventario, hoy he implementado dos funcionalidades visuales clave:

- **Efecto Lightbox:** He programado un sistema para que, al clicar en la miniatura de un artículo, la imagen se amplíe en el centro de la pantalla con un fondo oscurecido y difuminado, permitiendo ver los detalles del objeto.
- **Carrusel Dinámico:** He creado un componente que intercambia automáticamente las dos imágenes de la tarjeta cada 5 segundos. Esto aporta dinamismo visual a la lista de colecciones sin penalizar el rendimiento.

Autor: Joaquín Bizcocho

4. Gestión avanzada de roles y plantillas oficiales

He establecido una distinción clara entre el usuario estándar y el administrador (admin).

- He creado un **Panel de Administración** exclusivo donde puedo monitorizar estadísticas globales (total de usuarios, categorías y artículos).
- He implementado un sistema de **Plantillas Oficiales**. Como administrador, he creado categorías "maestras" que sirven de base para el resto de usuarios.

5. Rediseño del flujo de creación

He modificado el proceso de creación de colecciones. Ahora, cuando un usuario quiere añadir una categoría, puede elegir entre usar una de mis plantillas oficiales o crear una personalizada desde cero. Además, he ajustado el filtrado del dashboard principal para que las plantillas del sistema no se mezclen con las colecciones privadas, manteniendo la interfaz limpia y enfocada en lo que realmente importa al coleccionista.

Con estos cambios, he conseguido que la aplicación pase de ser un simple CRUD a una plataforma de coleccionismo inteligente y organizada. Todo el sistema de gestión de datos está funcionando correctamente y la navegación es fluida.

Fecha: 14 de abril de 2026

Durante la sesión de hoy, he pausado el desarrollo de nuevas funcionalidades para abordar las correcciones de seguridad indicadas en la última revisión del proyecto. El objetivo principal ha sido establecer un control de acceso real en el lado del servidor y proteger la información sensible.

Las tareas realizadas se han dividido en los siguientes puntos:

1. Gestión segura de credenciales

He eliminado las credenciales de la base de datos en texto plano del código fuente. Se ha modificado el archivo `application.properties` para que la conexión a la base de datos se realice mediante variables de entorno (`${DB_USER}`, `${DB_PASSWORD}`). Para facilitar el despliegue a terceros y mantener la documentación del repositorio, he creado el archivo `application.properties.example` a modo de plantilla genérica.

2. Autenticación y validación mediante JWT

Para dejar de depender exclusivamente de la lógica del frontend en el control de acceso, he integrado Spring Security en el backend utilizando JSON Web Tokens (JWT). Se ha desarrollado una utilidad para generar los tokens en el momento del login y un filtro (`JwtFilter`) que intercepta todas las peticiones entrantes para validar la firma y caducidad del token antes de permitir el acceso a los controladores.

3. Control de acceso basado en roles (RBAC)

En la clase de configuración de seguridad, se ha aplicado una política restrictiva. Únicamente los *endpoints* de registro y login permanecen públicos. El resto de la API requiere autenticación, y se ha añadido una capa extra para las rutas de administración (`/api/admin/**`), las cuales ahora devuelven un error 403 (Forbidden) si el token del usuario no contiene explícitamente el rol de administrador. Adicionalmente, se ha configurado la política CORS para permitir la recepción de cabeceras de autorización.

4. Adaptación del cliente web (React)

Una vez asegurado el backend, he refactorizado las llamadas a la API en el frontend. El componente de inicio de sesión se ha actualizado para almacenar el token en el almacenamiento local del navegador. Posteriormente, he revisado los controladores de las vistas de categorías y artículos para incluir la cabecera `HTTP Authorization: Bearer` en todas las peticiones asíncronas (`fetch`) de lectura y escritura.

Autor: Joaquín Bizcocho

5. Depuración y actualización documental

Durante las pruebas de integración, he solucionado varios errores de acceso denegado asegurando que las funciones de carga automática (useEffect) enviaran correctamente el token. También se ha corregido la asignación de variables en el formulario de creación de artículos. Por último, he actualizado la documentación operativa del proyecto para reflejar la nueva arquitectura de seguridad y los requisitos de arranque con variables de entorno.

Fecha: 30 de abril de 2026

Objetivo: Integración total del sistema, plan de pruebas de estrés y despliegue del sistema de recuperación ante desastres.

1. Integración del "Conjunto Coherente"

Hoy el objetivo no era que las piezas funcionaran solas, sino que bailaran juntas. He logrado integrar el servicio de **JavaMailSender** con el flujo de seguridad de **Spring Security**. Ahora, el sistema no solo registra un usuario, sino que bloquea su acceso mediante un filtro de autorización hasta que el token OTP enviado al correo es validado. He verificado que la jerarquía de componentes en React sea estable, asegurando que el estado de autenticación se propague correctamente desde la raíz (`App.jsx`) hasta los niveles más profundos como la gestión de artículos.

2. Plan de Pruebas y Resultados Documentados

He diseñado y ejecutado un plan de pruebas para estresar el sistema. Los casos de prueba incluyeron:

- **Inyección de datos corruptos:** Intenté guardar artículos con esquemas JSON malformados; el backend respondió correctamente con un `400 Bad Request`.
- **Prueba de concurrencia:** Verifiqué que el sistema de tokens JWT no colisionara al tener varias sesiones abiertas con distintos roles (Admin y User) en el mismo navegador.
- **Validación de Roles:** Confirmé que un usuario con rol user recibe un `403 Forbidden` inmediato si intenta acceder por URL manual a la ruta `/api/admin/estadisticas`.

3. Pulido de UX: El fin de los "datos fantasma"

He eliminado la incertidumbre visual en el panel de control. Antes, las métricas globales mostraban un "0" inicial que podía confundir al administrador. Ahora, he implementado un estado de carga dinámico (`cargandoStats`) con una animación de pulso en CSS. Esto no es solo estética; es **feedback informativo**: el usuario sabe que el sistema está consultando la base de datos y no que la colección está vacía.

Autor: Joaquín Bizcocho

4. Registro de Incidencias: Los errores más críticos y extraños

En esta fase final de integración, me he enfrentado a errores que casi me vuelven loco por lo específicos que eran:

- **El misterio del Token "mentiroso" (Error 403 persistente):** Fue el más raro. Cambiaba el rol de un usuario a admin en la base de datos, pero el servidor seguía dándole error 403. Descubrí que el **JWT (Token)** se genera con la información de ese momento exacto. Si no cierras sesión y vuelves a entrar, el token sigue diciendo que eres "user" aunque en la base de datos seas Dios. La solución fue forzar un `localStorage.clear()` al detectar un cambio de permisos.
- **La "Multiplicación" de artículos (Error de Lógica en PUT):** Al editar, el sistema creaba un duplicado en lugar de actualizar. El fallo era invisible en la consola: el JSON que enviaba React no incluía el campo `id`, por lo que Hibernate, al ver un objeto sin ID, decidía que era un nuevo registro. Fue añadir `id: idEditando` y el sistema volvió a ser estable.
- **El bloqueo de seguridad de Google (SMTP Error 535):** Spring Boot se negaba a enviar correos dando un error de autenticación a pesar de que la contraseña era correcta. Resultó ser el sistema de seguridad de Google para "apps menos seguras". Tuve que activar la verificación en dos pasos y generar una **contraseña de aplicación** específica para que Java pudiera entrar.
- **Conflicto de Nombres en LocalStorage:** Al cambiar de un hito a otro, React buscaba `collectorhub-token` mientras que el nuevo Login guardaba solo `token`. Esto provocó que la pantalla se quedara en blanco porque `App.jsx` encontraba un valor nulo y "explotaba". He unificado todas las claves de almacenamiento bajo una nomenclatura simple y coherente.

Fecha: 2 de mayo de 2026

Objetivo: Consolidación de la documentación técnica, despliegue de la infraestructura de pruebas y cierre del hito.

1. Implementación de Infraestructura de Pruebas Unitarias

Se ha configurado el entorno de pruebas en el backend, definiendo la estructura de directorios `src/test/java` y ajustando el gestor de dependencias Maven (`pom.xml`) para integrar **JUnit 5 (Jupiter)** mediante `spring-boot-starter-test`.

2. Desarrollo de Pruebas de Dominio

Se ha desarrollado la suite `UsuarioTest.java` para validar la lógica interna del modelo de datos de forma aislada, sin acoplamiento a la base de datos MySQL. Las pruebas garantizan que la asignación de propiedades críticas (como los roles de acceso) funciona correctamente, validando casos de frontera como la no asignación accidental de privilegios de administrador por defecto.

3. Resolución de Incidencias de Entorno

Durante la configuración de las pruebas, surgió una incidencia de dependencias donde el entorno de desarrollo (IntelliJ) no reconocía las anotaciones `@Test`. El problema derivaba de una desincronización en el ciclo de construcción de Maven. Se solucionó forzando una recarga completa del árbol de dependencias (`Reload All Maven Projects`), lo que permitió la correcta indexación de la librería y la ejecución exitosa de la batería de pruebas.

4. Plan de Contingencia y Validación

Se ha finalizado la redacción de la memoria técnica del proyecto. Siguiendo los requisitos del hito, se ha documentado exhaustivamente el **Disaster Recovery Plan**, detallando los comandos precisos para el volcado lógico (`mysqldump`) y la restauración desde contenedores Docker. Se ha integrado la matriz del plan de pruebas manuales, dejando el proyecto estabilizado, documentado y listo para su despliegue y evaluación.

Autor: Joaquín Bizcocho

Fecha: 5 de mayo de 2026

Objetivo: Resolución de incidencias de QA, sincronización de entornos para simulacros de contingencia y configuración del despliegue en la nube.

1. Pruebas de Integración y Refactorización del Backend

Se han desarrollado cuatro suites de pruebas automatizadas para los controladores principales del sistema (AdminController, ArtículoController, CategoriaController, AuthController). Se ha implementado la arquitectura de pruebas utilizando MockMvc para simular peticiones HTTP y la anotación @MockitoBean, estándar en Spring Boot 3.4.x, para aislar los servicios dependientes. Las pruebas garantizan la correcta validación de la lógica de negocio restrictiva, asegurando la denegación de acceso (HTTP 403) a usuarios con la cuenta inactiva o sin el PIN verificado.

2. Ejecución de Pruebas Manuales (E2E)

Se ha configurado el entorno cliente mediante Postman para validar el flujo real contra el servidor local. Se han documentado cinco escenarios de prueba que abarcan tanto flujos de éxito como de error (Happy Path y Sad Path). Esto incluye la correcta generación y captura del token JWT como Bearer Token en el endpoint de inicio de sesión, así como su posterior uso para la validación y autorización en rutas protegidas. Las evidencias visuales resultantes se han extraído para su inclusión en la memoria técnica.

3. Preparación del Simulacro de Contingencia (Disaster Recovery)

Se ha extraído el modelo DDL exacto directamente desde las entidades JPA del proyecto para generar un script de inicialización que clona la estructura de producción en el entorno local. Se ha configurado este entorno localizado utilizando contenedores Docker (mysql:8.0) y se ha adaptado el gestor de propiedades de Spring Boot. El simulacro físico de destrucción y recuperación de base de datos para validar el RTO y RPO ha quedado preparado y programado para su ejecución externa, debido a las restricciones operativas y de cortafuegos de la red corporativa actual.

4. Gestión de Despliegue y Migración de Infraestructura

Se ha iniciado la fase de puesta en producción conectando el repositorio de control de versiones con plataformas PaaS. Durante el proceso en la plataforma Railway, se detectó una incidencia crítica de compatibilidad y codificación estricta (MalformedInputException) relacionada con caracteres UTF-8.

Autor: Joaquín Bizcocho

Fecha: 06 de mayo de 2026

Hoy he enfocado mis esfuerzos en blindar la seguridad del backend, perfeccionar la adaptabilidad móvil y asegurar la integridad de los datos desde el cliente, dejando la arquitectura lista para el Beta Testing.

Estos son los avances de la jornada:

1. Implementación del patrón DTO y seguridad

He rediseñado la capa de presentación del servidor aislando el modelo de base de datos. Con esto, he mitigado vulnerabilidades críticas y evitado la exposición accidental de datos sensibles (contraseñas, pines) en las respuestas JSON.

2. Adaptación Mobile-First y navegación

He garantizado una visualización perfecta en móviles usando Media Queries para reorganizar las tarjetas con Flexbox. Además, he solucionado un molesto bloqueo de desplazamiento ("Scroll Lock") ajustando el CSS global para usar alturas relativas.

3. Coherencia visual y limpieza de layouts

He refactorizado el enrutador principal para eliminar márgenes no deseados. Ahora el Dashboard luce un diseño natural "Edge-to-Edge" (anclado a los bordes) y la barra de navegación tiene un estilo completamente homologado.

4. Mejora de micro-interacciones

He pulido la UX táctil eliminando el efecto "Sticky Hover" en móviles. También he personalizado el diseño nativo de los botones de subida de archivos para adaptarlos a la marca, y he protegido las tarjetas para que no se rompan con textos demasiado largos (usando truncado o *ellipsis*).

5. Integridad de datos desde el cliente

He añadido validaciones lógicas en el frontend para evaluar si una categoría está vacía antes de llamar a la API. Así bloqueo la creación de "categorías fantasma", evito peticiones inútiles al servidor y garantizo que la base de datos solo reciba información válida.

Con estos cambios, la plataforma es mucho más robusta a nivel interno y ofrece una experiencia móvil y visual completamente profesional.

Autor: Joaquín Bizcocho

Fecha: 07 de mayo de 2026

Hoy me he centrado de lleno en estabilizar la aplicación en producción (Render y Vercel), solucionar los bloqueos de seguridad y arreglar por fin el sistema de envío de correos, que me estaba dando bastantes dolores de cabeza en la nube.

Estos son los avances de la jornada:

1. Mejora de la interfaz y validación de inputs

He solucionado un problema visual donde los textos demasiado largos rompían las tarjetas y el diseño. Para evitarlo, he aplicado reglas CSS de control de desbordamiento (`overflow-wrap`) y añadí un límite máximo de 50 caracteres directamente en React. También he puesto una validación extra que bloquea el envío del formulario si faltan campos obligatorios.

2. Resolución de bloqueos de seguridad (CORS y error 403)

Me ha tocado pelearme un poco con Spring Security. El servidor bloqueaba las peticiones (error 403) porque el filtro CSRF chocaba con mis tokens JWT, así que lo he desactivado para la API. Además, arreglé la política de CORS (una simple barra / al final de la URL de Vercel me estaba tirando la conexión) y ajusté el filtro JWT para que deje entrar libremente a las rutas de registro y verificación de PIN.

3. Migración del sistema de correos a EmailJS

El protocolo SMTP de Gmail fallaba constantemente en los servidores de Render por temas de puertos. La solución ha sido cambiar esa lógica y conectarme a la API REST de EmailJS. He creado una plantilla dinámica y ahora mi backend manda los datos de forma asíncrona (`@Async`). El cambio ha sido brutal: ahora los correos llegan al instante y no fallan.

4. Limpieza de "usuarios fantasma"

Detecté que si el envío del correo fallaba, la cuenta se quedaba guardada a medias en la base de datos, impidiendo que la persona pudiera volver a intentar registrarse con ese mismo correo o alias. He modificado el controlador para que, si el mail da error, se haga un borrado (`delete`) inmediato de esa cuenta temporal. Así mantengo la base de datos limpia de registros a medias.

Fecha: 10 de mayo de 2026

Autor: Joaquín Bizcocho

Objetivo: Ejecutar de verdad el simulacro de backup y recuperación que tenía documentado pero sin probar.

1. Problemas arrancando Docker

Al intentar lanzar el contenedor me encontré con dos problemas seguidos: primero Docker Desktop no estaba corriendo, y cuando lo abrí y volví a intentarlo me dijo que ya existía un contenedor `ch_database` de antes ocupando el nombre. Lo borré con `docker rm -f ch_database` y ya arrancó bien en el puerto 3307.

2. Metiendo datos de prueba

Como el backend seguía apuntando a Railway no quería tocarlo, así que inserté los datos directamente con SQL desde la terminal. PowerShell me dio guerra con las comillas del JSON así que acabé tirando de un archivo `.sql` intermedio y pasándoselo al contenedor con `Get-Content`. Al final metí 1 usuario, 2 categorías y 2 artículos.

3. Generando el backup

Lancé el `mysqldump` y se generó el archivo `backup_diario.sql` sin problemas. Pesaba 9.520 bytes y al abrirlo en VS Code se veía el SQL completo con los `CREATE TABLE` y los `INSERT INTO`, incluyendo los campos JSON.

4. Destruyendo la base de datos

Aquí viene la parte importante. Usé `docker-compose down -v` para eliminar también el volumen, no solo el contenedor. Así me aseguré de simular una pérdida real de datos y no hacer trampa borrando solo los registros. Al volver a levantar Docker y ejecutar los conteos: `usuarios=0`, `categorias=0`, `articulos=0`.

5. Restaurando y verificando

Importé el backup con `Get-Content backup_diario.sql | docker exec -i ch_database mysql` y el comando terminó sin errores. Los conteos después de restaurar: `usuarios=1`, `categorias=2`, `articulos=2`. Exactamente igual que antes del borrado, con los JSON intactos.

Estado del proyecto: Simulacro completado y documentado con capturas de cada paso. La documentación operativa ya tiene la sección 8 actualizada con las evidencias reales. Listo para entregar.

Fecha: 12 de mayo de 2026

Objetivo: Revisión y refactorización de la arquitectura del proyecto para mejorar la seguridad, la calidad del código y la mantenibilidad general.

1. Seguridad en el backend: credenciales y autorización

He resuelto dos problemas de seguridad importantes. Por un lado, las claves de EmailJS estaban hardcodedas en el código; las he migrado a variables de entorno con `@Value` de Spring, dejando los valores reales solo en el panel de Render. Por otro, los controladores confiaban en el `usuarioId` que mandaba el frontend por la URL, lo que permitía a cualquier usuario ver o borrar recursos ajenos. He creado la clase `AuthenticatedUser` y modificado el `JwtFilter` para que el id del usuario se extraiga siempre del token JWT. Los métodos PUT y DELETE ahora devuelven 403 si el recurso no es tuyo. De paso, también he sustituido `Random` por `SecureRandom` en la generación del PIN, que es lo correcto para operaciones de seguridad.

2. Validación de DTOs y capa de servicios

He añadido `spring-boot-starter-validation` al `pom.xml` y decorado los DTOs con anotaciones como `@NotBlank`, `@Size` y `@NotNull`, de forma que el backend rechaza automáticamente con un 400 cualquier petición con datos inválidos. Además, he introducido la capa de servicios: he creado `CategoriaService.java` y `ArticuloService.java` moviendo toda la lógica de negocio fuera de los controladores. Ahora la arquitectura sigue correctamente el patrón `Controller → Service → Repository`, con cada capa teniendo una única responsabilidad.

3. Centralización de la API y manejo de sesión expirada en el frontend

Tenía la URL de Render hardcodeda en más de 30 llamadas `fetch` distintas. He creado `src/services/api.js` con todos los métodos organizados (`categoriasApi`, `articulosApi`, `adminApi`) y la URL base leyendo de variables de entorno `Vite`. He añadido también un wrapper `fetchConAuth` que intercepta cualquier 401 o 403, limpia el `localStorage` y redirige automáticamente al login con un mensaje explicativo, en vez de dejar al usuario con errores sin sentido.

4. Mejoras de UX y corrección de bugs

He resuelto varios problemas de experiencia de usuario: el input de subida de imágenes tenía el estilo feo nativo del navegador y lo he sustituido por un botón

Autor: Joaquín Bizcocho

personalizado. El botón de registro no se bloqueaba mientras esperaba respuesta del servidor, lo que con Render frío provocaba que se crearan cuentas duplicadas al hacer varios clics; ahora se deshabilita y muestra "Registrando..." durante la petición.

5. Incidencias y cosas aprendidas

Durante la sesión me topé con varios quebraderos de cabeza. El más molesto fue el problema de tipos genéricos de Java al usar `.map()` con lambdas en los métodos DELETE: Java no conseguía inferir `ResponseEntity<Void>` correctamente y daba error de compilación. Lo resolví reescribiendo esos métodos con `if/return` directo en vez de encadenamiento funcional, lo que además quedó más legible. También aprendí a diferenciar bien cuándo usar `@NotBlank` frente a `@NotNull` en los DTOs, ya que `@NotBlank` solo aplica a Strings y rechaza también cadenas vacías o con solo espacios, mientras que `@NotNull` simplemente comprueba que el campo no sea nulo. Cosas que parecen pequeñas pero que si las confundes te dan validaciones que no funcionan como esperas.

Fecha: 13 de mayo de 2026

Objetivo: Implementación de funcionalidades pendientes de la planificación original, refactorización del código frontend, corrección de pruebas automatizadas y mejoras generales de calidad y usabilidad.

1. Exportación e Importación de Datos (CSV y JSON)

He desarrollado los endpoints de exportación en el backend para permitir la descarga de la colección de una categoría en formato CSV y JSON. He implementado la funcionalidad simétrica de importación, incluyendo detección de conflictos que avisa al usuario cuando los artículos del fichero ya existen, solicitando confirmación antes de sobrescribirlos. He añadido validación en el backend para rechazar ficheros con formato incorrecto. En el frontend he agrupado ambas funcionalidades en un único desplegable "Datos ▼" para no saturar la cabecera.

2. Wishlist y Panel de Estadísticas

He añadido el campo estado a la entidad Artículo (COLECCION / WISHLIST) con su correspondiente actualización en el SQL, DTO y servicio. En el frontend he implementado dos pestañas que filtran los artículos por estado. Paralelamente he añadido un panel lateral de estadísticas que muestra en tiempo real el conteo de artículos, la suma o media de campos numéricos con toggle entre ambos modos, y el conteo de valores Sí/No para campos booleanos, todo calculado en el cliente sin peticiones adicionales al servidor.

3. Borrado Completo de Cuenta (RGPD)

He implementado el endpoint DELETE /api/usuario/cuenta en un nuevo UsuarioController. Gracias a las restricciones ON DELETE CASCADE del esquema SQL, el borrado elimina en cascada todas las categorías y artículos del usuario. En el frontend he añadido un botón discreto con doble confirmación y limpieza automática del localStorage tras el borrado.

Autor: Joaquín Bizcocho

4. Refactorización del CSS y Variables de Entorno

He dividido el fichero `Categorias.css` en cuatro ficheros independientes: `Categorias.css`, `Articulos.css`, `Admin.css` y `Responsive.css`, de forma que cada componente importa únicamente sus estilos correspondientes. He centralizado la URL del backend en la variable de entorno `VITE_API_URL` configurada en Vercel, eliminando la URL hardcodeada del código fuente.

5. Corrección de Tests y Mejoras de Usabilidad

He corregido los tests de `AuthControllerTest` y `ArticuloControllerTest` que fallaban por incompatibilidad entre el formato de respuesta del backend y las aserciones del test, añadiendo además la dependencia `spring-security-test` para inyectar correctamente el usuario autenticado. He mejorado los mensajes de error del login para mostrar textos específicos según el código HTTP. He limitado con `maxLength` los campos del esquema para evitar desbordamientos visuales y corregido el scroll horizontal en el modal de edición de artículos.

Fecha: 15 de mayo de 2026

Objetivo: Redacción y cierre de la documentación final del proyecto.

1. Documentación técnica completa

He dedicado el día entero a cerrar la documentación del proyecto. Empecé estructurando todos los apartados que necesitaba cubrir según la rúbrica de evaluación y fui redactando cada uno: el modelo de datos explicando las tres entidades y sus relaciones, la arquitectura del backend con el patrón Controller → Service → Repository, el sistema de seguridad JWT con los fragmentos de código más relevantes, las funcionalidades más complejas como el flujo de verificación por PIN y la importación con detección de conflictos, y toda la parte del frontend con la estructura de componentes y el api.js centralizado.

2. Tabla de endpoints y revisión general

Añadí la tabla completa con todos los endpoints de la API indicando método, ruta, si requiere autenticación y descripción. Aproveché también para revisar que cada apartado cubría los puntos de la rúbrica y reordenar algunas secciones para que el documento tuviera más coherencia.

3. Planificación preliminar vs real y lecciones aprendidas

Redacté la comparativa entre lo que planifiqué al inicio del proyecto y lo que realmente ocurrió, incluyendo las desviaciones más importantes como la migración a Railway, la verificación por PIN o los DTOs que añadí casi al final. También escribí las lecciones aprendidas con lo que repetiría, lo que no volvería a hacer y las tres mejoras que aplicaría en una versión 2.

4. Portada y cierre

Diseñé la portada del documento con el logo del proyecto y los datos del ciclo. Eliminé el apartado de próximos pasos que había quedado del borrador y que no tenía sentido en la entrega final. Con esto el documento queda cerrado y listo para entregar junto al enlace de la aplicación desplegada en Vercel y el repositorio de GitHub.

Autor: Joaquín Bizcocho